Privilege, setuid and sudo

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-19

setuid and sudo: Lecture Overview

The Privilege Model in Motion: how a process comes to hold more privilege than the user who started it

sudo as Authorisation Policy: granting by rule instead of by ownership, and why one command is rarely one capability

The Attacker's Foothold: who we are modelling, and which link of the capstone this is

Into the Lab: two legitimate mechanisms, used carelessly, found and closed from an unprivileged shell

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

Real, Effective, Saved, in Motion

- Week 3 gave you the vocabulary. Today it moves: those three numbers change during a process's life
- **Real** is who started it. **Effective** is what it counts as now. **Saved** is what it may go back to
- The interesting security events are exactly the moments one of them changes
- And the answer to 'how did this process become root' is always one of a very small number of transitions

Three numbers, one question: which of them was the kernel reading when it said yes?

The Kernel Checks the Effective UID

```
$ id -ru        # real: who you are
1000
$ grep ^Uid: /proc/self/status
Uid:  1000  1000  1000  1000
      real  eff   saved fs

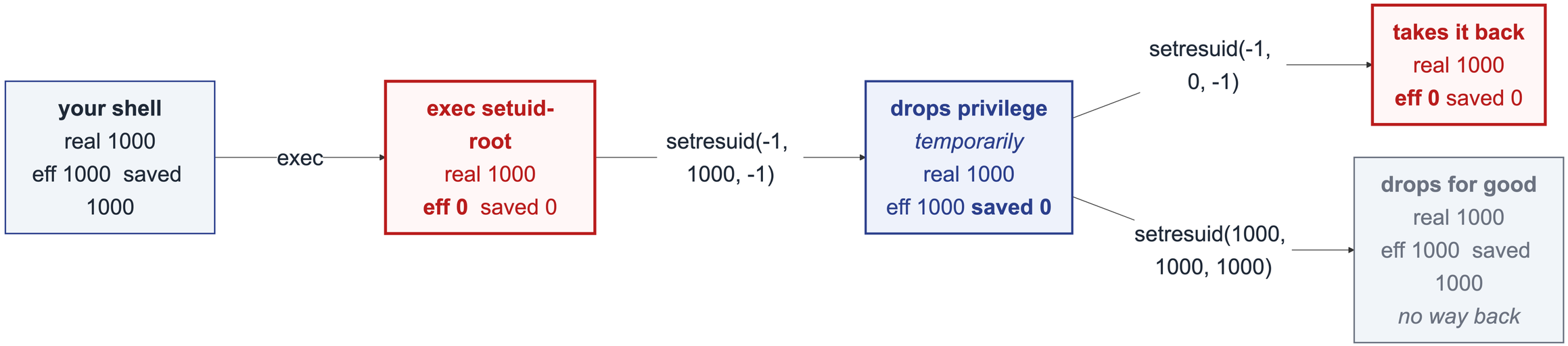
# access decisions normally read the
# second number, the effective UID
```

- Reading a root-only file: the check reads the **effective** UID
- The real UID is for accountability and for deciding who may signal whom. It is not what grants access
- With setuid and sudo, **becoming root** means getting the effective UID to 0
- This sentence is what Lab 6 asks you to prove, from an account that must not be able to

Real is who you are. Effective is what you count as. The second one is what grants access.

How a Process Changes Identity

- On exec of a **setuid** binary, the effective and the saved UID become the file's owner. That is the whole mechanism
- A careful program then **drops** privilege, keeping it in the saved UID so it can be taken back for one operation
- Dropped is not gone: while the saved UID holds 0, the process is one call from root



A process that has dropped privilege is not unprivileged. It is a process that has decided not to use it yet.

The Bit on the Binary

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root ... /usr/bin/passwd
    ^
    the s: on exec, effective UID becomes
    the file owner, which here is root
```

- `passwd` is the honest example: it must write `/etc/shadow`, which you cannot write
- So the privilege is attached to the **program**, not to you, and it lasts only while that program runs
- This is a real solution to a real problem, and it predates every other mechanism in this unit
- The trouble is what it grants: not one file, but everything `root` can do

Show the mechanism working before you see it misused. passwd needs exactly this, and gets far more than it needs.

Why setuid Is a Design Smell

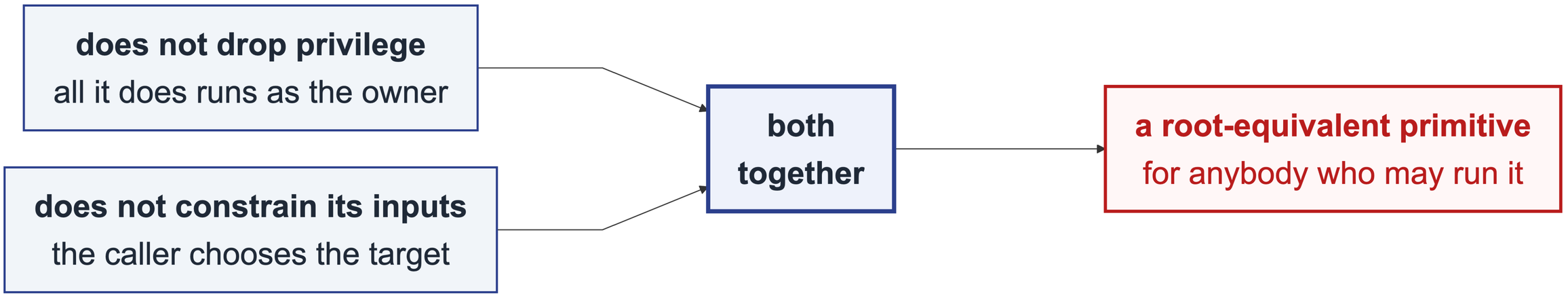
- The grant is **all of the owner's privilege**, held for the life of the process unless it deliberately gives it up
- So every setuid binary is a promise that its author handled that gap correctly, in every code path, for ever
- The blast radius is the reason Week 7 exists: capabilities let you grant the sliver instead of the whole



Not a bug: a grant far larger than the need, repeated on every setuid binary on the system.

The Failure Pattern

- Two properties, and a setuid binary is a root-equivalent primitive for anybody who may run it
- Neither alone is fatal. Together they hand the caller the owner's identity, in whatever shape the tool provides



Ask two questions of any setuid binary: does it drop, and does it let me choose what it touches?

Finding Them

```
$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/mount
/usr/local/bin/<a local tool>
...
```

- One command finds the setuid binaries on a host. It belongs in your first ten minutes there
- Most are the distribution's: packaged, audited, old enough to have been looked at
- A **custom** one deserves more suspicion: nobody audited it, and somebody put it in `/usr/local`
- You already met this one in Week 1. This week you find out what it is for

The list is short and the odd one out is usually obvious. Knowing to look is the whole skill.

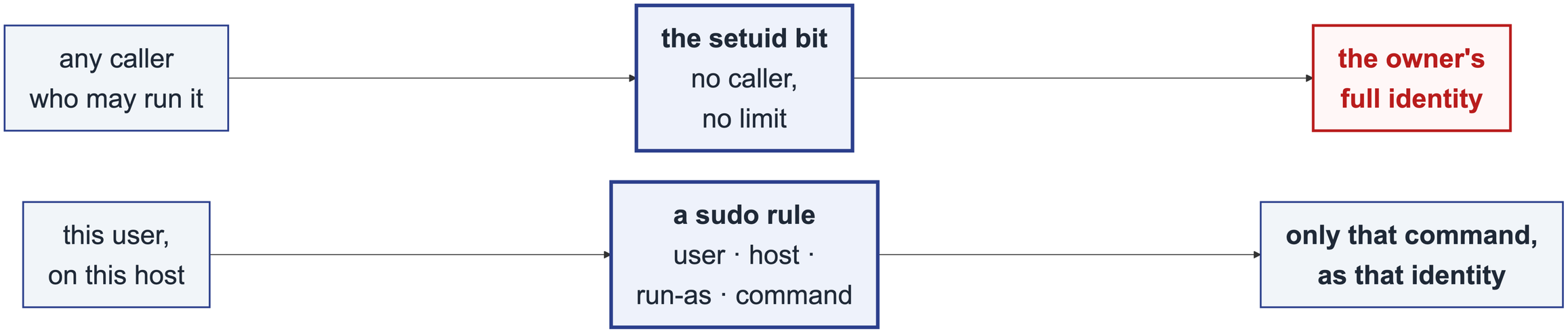
Lab Core

sudo as Authorisation Policy

A file bit grants everything its owner has. What would it take to grant only part?

From a File Bit to a Rule

- So it can be narrow in a way a file bit cannot, and it can be audited by reading a file
- It is also the first authorisation policy in this unit that somebody has to write, and therefore can get wrong



The mechanism is better. The failure mode moves from the program's code to the administrator's sentence.

The sudoers Model

```
student ALL=(ALL) ALL
  ^user   ^host ^runas ^command

svc-report ALL=(ALL) NOPASSWD: /usr/bin/find

# drop-ins, read in order:
#   /etc/sudoers  then  /etc/sudoers.d/*
```

- Four fields, and every one of them is a decision: who, where, as whom, and what
- `NOPASSWD` removes the one control that slows an attacker who already has a shell
- Drop-in files add rules without editing the main file, and are how rules get forgotten
- The second line is on your machine. You read it as the account itself in the lab

Read every rule as a sentence. If any field is ALL, say the sentence out loud and hear what it grants.

Read Your Own Policy

```
$ sudo -ln
User svc-report may run the following
commands on this host:
  (ALL) NOPASSWD: /usr/bin/find
```

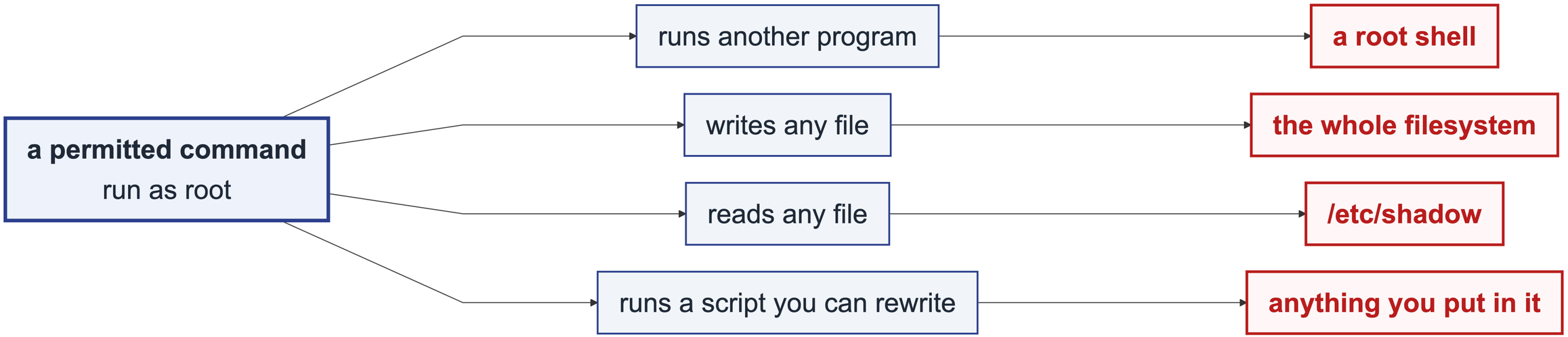
- `sudo -ln` prints what **you** are allowed, and it needs no privilege beyond being the account
- So it is the first thing an attacker runs after landing, and the first thing a defender should check
- It answers a question the file cannot: not what the policy says, but what it grants **to this account**
- The answer above is one line, and it is enough

Lab Core

An attacker's first minute on a host: id, sudo -ln, find -perm -4000. All three are free.

One Command Is Rarely One Capability

- The rule names a **command**. What it grants is everything that command can be made to do
- So the question is never 'do I trust this command'. It is 'what is the closure of what this command can reach'



Granting a command grants its capabilities, not its usual purpose. Most rules are written against the purpose.

Three Ways a Grant Escapes

```
# a permitted find: its -exec runs
# any program it is told to, as root

# a permitted pager or editor: shell escape
$ sudo less /var/log/syslog   ->  !sh

# a permitted script you can write
$ echo 'id' >> /opt/maint/rotate.sh
```

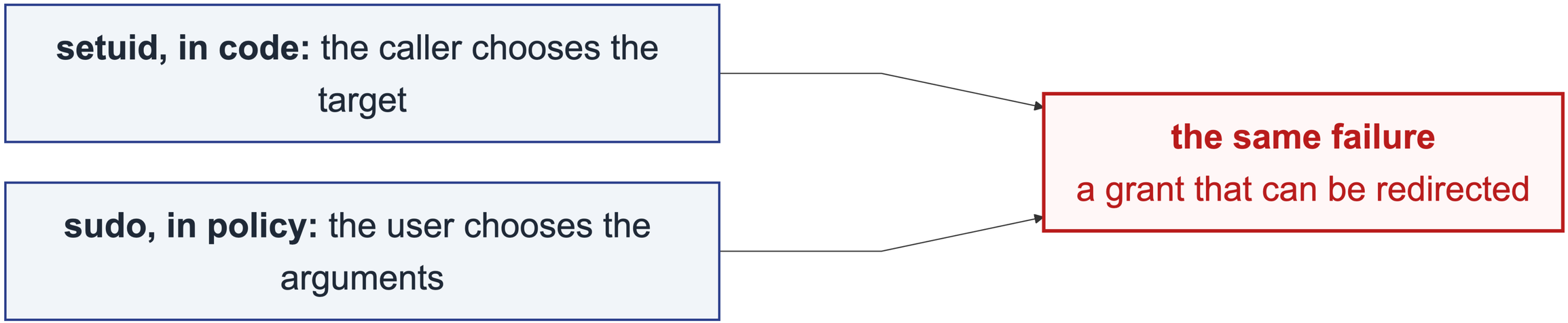
- `find`'s `-exec` is not an obscure feature: it is the reason `find` exists
- A pager is worse than it looks: `less` and `man` both drop to a shell by design
- And a script in a directory you can write is the Week 1 finding, arriving again wearing a sudo rule
- **GTFOBins** catalogues these for hundreds of binaries. Attackers look them up, not invent them

None of these is an exploit. Every one is the tool's documented behaviour, invoked as root.

Lab Core

The Same Shape as setuid

- Both mechanisms grant privilege without constraining what the privilege can be turned into
- setuid does it in code: the program holds the owner's identity and lets the caller choose the target
- sudo does it in policy: the rule holds root and lets the user choose the arguments
- One lesson, two places: **the grant is safe only if the thing granted cannot be redirected**



Recognising the shape is worth more than memorising either mechanism. It recurs in Blocks 4 and 5.

Fixing It

```
# before: a command, and its whole closure
svc-report ALL=(ALL) NOPASSWD: /usr/bin/find

# after: scope to the exact operation the
# account needs, arguments and all, as root,
# or decide the need does not require root
```

- **Scope the command** to the operation, arguments included, rather than to the tool
- **Remove the escape** where you can: a rule permitting a pager or an editor is almost never safe
- **Or decide it cannot be granted**, and solve the problem another way. That is a legitimate answer
- Narrow the runas too: `(root)` rather than `(ALL)` says who they may become

The fix is not a better command. It is a smaller grant, written against the operation somebody actually needs.

Least Privilege, Again

- Both mechanisms are safe **exactly when the grant equals the need**, and dangerous by the size of the gap
- setuid's gap is structural: the mechanism cannot express anything smaller than all of the owner
- sudo's gap is editorial: the mechanism can be narrow, and the rule usually is not
- Week 7 closes the structural one. Nothing closes the editorial one except reading the rule as a sentence

One mechanism cannot be narrow. The other can, and is not. Different problems, different fixes.

The Attacker's Foothold

You have sudo on your own machine. So what exactly are you proving in the lab?

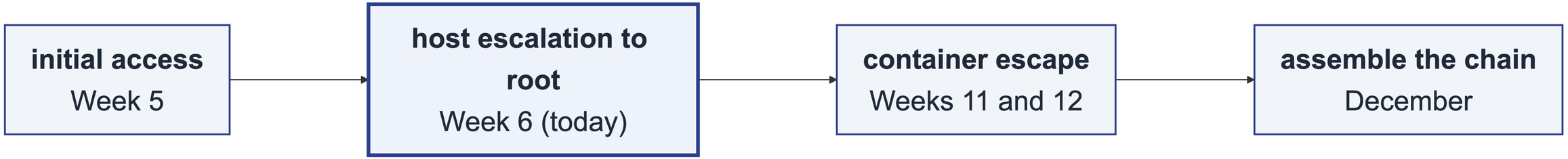
Not the Owner

- The model has been the same since Week 2: an **unprivileged non-owner** foothold, concretely `svc-report`
- `student` is a full administrator by design, so an escalation performed as `student` proves nothing at all
- In the lab you drop to the service account first, and every exploitation step happens from there
- It is also the realistic case: intrusions arrive as a service account, not as the person who owns the laptop

If you can already do it, you have not escalated. Drop to svc-report before you claim anything.

Capstone Link 2

- In December the target machine ships one of these two weaknesses, on the host side
- You will arrive there with a foothold, and the escalation to `root` is this hour's material, unchanged
- So today's laboratory is not a rehearsal for the capstone. It is the capstone's second link, taught early
- The other links: initial access was Week 5, and the container route is Weeks 11 and 12



Four links, taught in the weeks they belong to. In December you assemble them, you do not learn them.

Capabilities Are the Answer

- Next week splits `root` into about forty separate powers, and lets you grant one
- `ping` needs to open a raw socket. Under setuid it gets all of root; with a capability it gets that one power
- So the least-privilege gap closes: the grant can finally be the size of the need
- And a new question opens, which is Week 7's real subject: which of the forty are equivalent to root anyway?

Splitting root is progress, not a solution. Some of the pieces are still the whole thing.

Today in the Laboratory

Every exploitation step happens from the svc-report shell. Snapshot lab6-clean before you start.

- Enumerate the setuid binaries and pick out the one that is not the distribution's
- Abuse it from the service account to obtain something that account must not have
- Read your policy with `sudo -ln`, then turn the permitted command into a root shell
- Then close both, and prove the closure from the same shell that opened them

Exploit, then fix, then re-run the exploit. The re-run is the part that proves the fix holds.

Lab Core

Legitimate Mechanisms, Used Carelessly

- Nothing in this hour was a kernel bug. Both mechanisms did exactly what they are documented to do
- The world-writable script in Week 1 was the same shape. So was the agent-forwarding default in Week 5
- Three weeks, three mechanisms working correctly, three escalation paths
- This is what systems security mostly is: the gap between what a mechanism grants and what somebody needed

Look for the gap, not for the bug. The gap is more common, easier to find, and just as fatal.

Reading and Reference

- **credentials(7), execve(2), setresuid(2), sudoers(5)**. The manual pages behind every claim in this hour. Read them on the machine.
- **GTFOBins**. The catalogue of what permitted binaries can be turned into. Look up find, less and vi. https://gtfobins.github.io/
- **W. Du, Computer Security: A Hands-on Approach, 3rd ed., 2022**. The set-UID chapter, which is the closest published treatment to today's laboratory.
- **Stallings and Brown, Computer Security: Principles and Practice, 4th ed., 2018**. Chapter 4 (Section 4.4, UNIX File Access Control) for the Unix access-control background.
- **Saltzer and Schroeder, 1975**. Least privilege, in the paper that named it. Two pages are enough. https://www.cs.virginia.edu/~evans/cs551/saltzer/

Reference

Key Takeaways

- Permission decisions read the **effective** UID. Escalation here means moving that one number
- setuid grants all of the owner, and a temporary drop is not gone: the saved UID keeps it one call away
- sudo can be narrow and usually is not: the grant is the **closure** of what the command can reach
- Prove escalation from an account that could not already do it, or you have proved nothing
- Neither mechanism is broken. The gap between grant and need is where the escalation lives

Two questions for any grant: what does it actually permit, and what can it be turned into?